

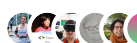
[← Назад к статьям](#)

Смешанная модель экспертов (МоЕ) в трансформерах

Опубликовано 26 февраля 2026 г.

[Обновление на GitHub](#)

▲ Отметить как понравившуюся **173**

 +167



Аритра Рой Гостхипати
[ariG23498](#) Подписаться



Педро Куэнка
[pkuenq](#) Подписаться



Мерве
[Мерве](#) Подписаться



Ильяс Мутаввакиль
[IlyasMoutawwakil](#) Подписаться



Артур Цукер
[ArthurZ](#) Подписаться



Серхио Паниего
[Серхиопаниего](#) Подписаться



Pablo Montalvo
[Молбап](#) Подписаться

🔗 Введение

За последние несколько лет масштабирование плотных языковых моделей привело к наибольшему прогрессу в LLM. Начиная с ранних моделей, таких как оригинальный [ULMFiT](#) (~ 30 миллионов параметров) или GPT-2 (1,5 миллиарда параметров, которые в то время считались "слишком опасными для выпуска" 🧙), и заканчивая современными системами с сотнями миллиардов параметров, рецепт был прост:

“Больше данных + больше параметров обеспечивают более высокую производительность.”

Законы масштабирования усилили эту тенденцию, но плотное масштабирование имеет практические ограничения:

- Обучение становится все более дорогостоящим.
- Растет задержка логического вывода.
- Для развертывания требуются значительные объемы памяти и аппаратного обеспечения.

Именно здесь на сцену выходят смеси экспертов (Mixture of Experts, MoE).

Если вы уже знакомы с MoE и хотите сразу перейти к инженерным решениям, используемым в трансформерах, можете сразу перейти к трансформерам и MoE.

🔗 От плотного к разреженному: что такое смеси экспертов?

Модель Mixture of Experts сохраняет основу Transformer, но заменяет некоторые плотные слои прямого распространения на набор **экспертов**. «Эксперт» — это не специализированный модуль (например, «эксперт по математике» или «эксперт по коду»). Это просто обучаемая подсеть. Для каждого токена **маршрутизатор** выбирает небольшую группу экспертов для его обработки.

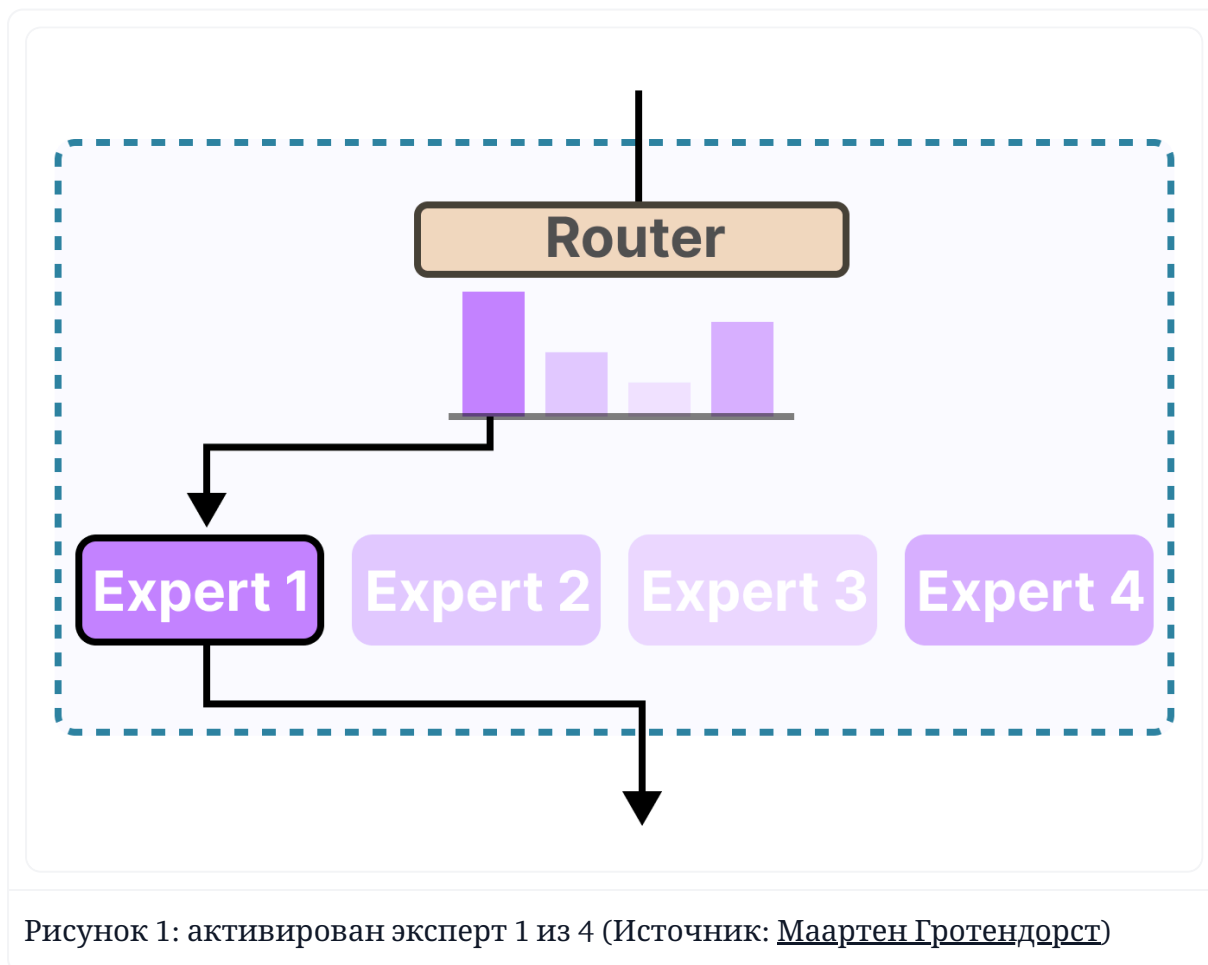


Рисунок 1: активирован эксперт 1 из 4 (Источник: [Маартен Гротендорст](#))

Разные токены активируют разных экспертов в зависимости от их скрытых представлений.

“Емкость модели зависит от общего количества параметров, но скорость логического вывода зависит от активных параметров.”

В этом и заключается ключевая идея.

Например, возьмем `gpt-oss-20b`. У нее 21 миллиард параметров, но на каждый токен приходится 4 активных эксперта из 32. С учетом общих компонентов и активных экспертов эта модель использует ~3,6 миллиарда активных параметров на токен. Если запустить эту модель на Mac M3 Ultra с пропускной способностью памяти около 800 ГБ, то скорость генерации можно оценить как $\sim 800 / (3.6 * 2)$ в bfloat16, где каждый параметр занимает 2 байта. Таким образом, получается около **111 токенов в секунду**. Фактическая производительность

составляет ~115 токенов в секунду, что очень близко к приблизительным расчетам.



0:00 / 0:08

Такая сверхвысокая скорость подтверждает, что модель работает примерно так же, как модель с 3,6 миллиардами параметров, но при этом обладает такой же производительностью (или качеством), как модель с 21 миллиардом параметров.

(Примечание: скорость была бы еще выше, если бы мы использовали ядра для нативного квантования `tf4`, которое используется в модели).

MoE привлекательны по следующим причинам:

1. Более высокая вычислительная эффективность

При фиксированном бюджете на вычисления с плавающей запятой глубокого обучения часто превосходит плотные аналоги.

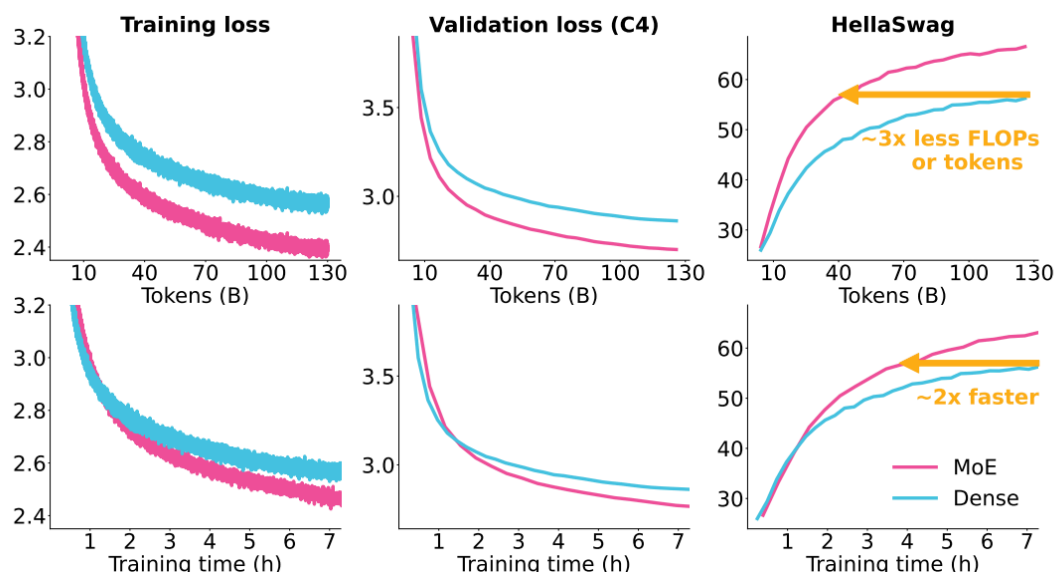


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both with the same configuration for 130B tokens. The MoE contains 64 experts out of which 8 are activated with an FFN dimension of 1,024, while the dense model has an FFN dimension of 8,192. Thus both have the same number of active parameters. **Top:** The MoE reaches the final dense performance with $\sim 3\times$ fewer tokens (or FLOPs, as both have the same active parameters ignoring the trivial router parameters). **Bottom:** Due to some memory overhead, this equates to $\sim 2\times$ faster training. More results, logs, and configurations: <https://wandb.ai/ai2-llm/olmoe/reports/Plot-MoE-vs-Dense--Vmlldzo40TMOMjkx>

Figure 2: Dense vs. MoE training curves (Source: [OLMoE: Open Mixture-of-Experts Language Models](#))

This means faster iteration and better scaling efficiency.

2. A Natural Parallelization Axis

Experts provide a structural boundary in the computation graph. Since different tokens engage different experts, we can parallelize across experts (we discuss this later in [Expert Parallelism](#)).

3. Industry Adoption

Recent major MoE releases of open models that happened in the past few weeks include [Qwen 3.5](#), [MiniMax M2](#), [GLM-5](#), or [Kimi K2.5](#).

The trend accelerated after the success of [DeepSeek R1](#) in January 2025, building on earlier systems like [DeepSeek V2](#). Another early MoE was [Mixtral-8x7B](#), released in December 2023.

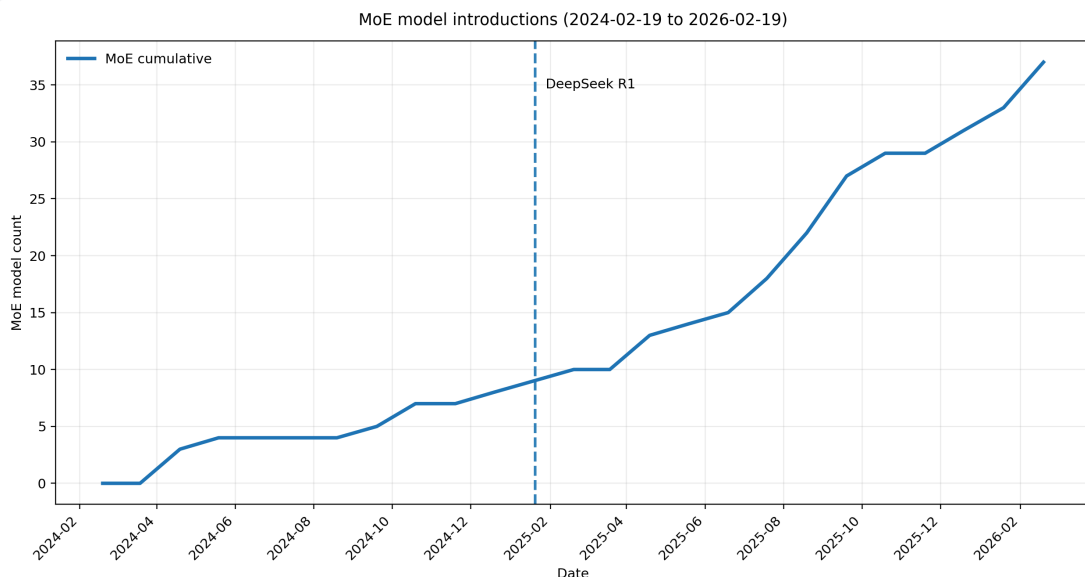


Figure 3: 2-year timeline of MoE model addition to the `transformers` library. DeepSeek R1 marks a clear inflection point.

Closed labs use MoEs too. ChatGPT has long been rumored to use a sparse architecture, and the open gpt-oss models certainly do.

If you want to learn more about MoEs in general, we strongly suggest reading [this blog](#) and watching our recent [YouTube video on routing](#).

🔗 Transformers and MoEs

Most tooling in the ecosystem, including model loading, device placement, quantization, and backend execution was originally designed for **dense** models. MoEs challenge these assumptions.

Making MoEs **first-class citizens** in `transformers` means redesigning parts of the loading pipeline, execution model, and distributed abstractions, not just adding new model classes. We'll focus on how the `transformers` library has evolved to support sparse architectures across:

- [Weight Loading Refactor](#)

- [Expert Backend](#)
- [Expert Parallelism](#)
- [Training MoEs with transformers](#)

🔗 Weight Loading Refactor

`AutoModelForCausalLM.from_pretrained("model_id")` downloads and loads model weights into a PyTorch model. For dense models, loading is relatively straightforward where each tensor in the checkpoint maps one-to-one to a parameter in the runtime module.

For MoEs, it's more complicated. In most MoE checkpoints, each expert is serialized independently. If you peek inside the [DeepSeek-V3 checkpoint index](#), you'll see keys like:

```
model.layers.3.mlp.experts.0.gate_proj.weight
...
model.layers.3.mlp.experts.255.gate_proj.weight
```

Each expert has its own set of weight matrices, essentially 256 (0 to 255 total, taking DeepSeek-V3 as an example) small feed-forward networks saved side by side. At runtime, however, GPUs execute optimized kernels. Modern MoE kernels such as [grouped GEMMs and fused MoE implementations](#) are designed to process *all experts in a single operation*, not by looping over them one at a time.

To do that efficiently, they require expert weights to be packed into a single **contiguous tensor**.

So we have a mismatch:

- **Checkpoint:** 256 separate tensors
- **Runtime:** 1 packed tensor

Bridging this gap systematically is what the [weight loading refactor](#) enables.

With the introduction of a generic WeightConverter, the mental model shifted from:

“A checkpoint already matches my runtime layout; loading is mostly a key-by-key copy.”

to:

*“A checkpoint is just a serialized source of tensors. Loading is a **conversion pipeline** that transforms them into the runtime layout we want.”*

🔗 Dynamic Weight Loading with `WeightConverter`

The central abstraction introduced by this refactor is **dynamic weight loading** via a WeightConverter.

`WeightConverter` lets us define:

```
source key patterns → target key(s) + operations
```

Primitive operations (chunk, concatenate, etc.) are composable. Two that are particularly useful for MoEs:

- MergeModuleList merges a list of tensors into a single tensor. For example, you can compose `MergeModuleList` with `Concatenate` to stack the experts in a MoE and pack them into one tensor.

```
WeightConverter(  
    ["block_sparse_moe.experts.*.w1.weight", "block_sparse_moe.experts.*.w2.weight",  
     "mlp.experts.gate_up_proj",  
    operations=[  
        MergeModuleList(dim=0),  
        Concatenate(dim=1),  
    ],  
)
```

- `SplitModuleList` splits a tensor back into a list of tensors. For example, you can split a stack of experts back into individual experts.

```
WeightConverter(  
    "mlp.experts.down_proj",  
    "block_sparse_moe.experts.*.w2.weight",  
    operations=[SplitModuleList(dim=0)],  
)
```

🔗 Lazy Materialization of Tensors

The refactor improves not just *what* conversions exist, but *how* they're scheduled.

The loader scans checkpoint keys once, matches them against converter patterns, and groups tensors per converter. Once a key is identified as needed, it's registered as a *future* and materialized via a thread pool. Conversion operations run only once their dependencies are ready. For example, `MergeModuleList` waits until all experts for a layer are loaded.

This avoids repeated scans and reduces memory peaks.

🔗 Benchmark: Weight-Loading Pipeline Improvements

To evaluate the improvements introduced by the new weight-loading pipeline, we benchmarked the v4 vs v5 versions of `transformers`. The focus is on loading speed of large MoE models, which is often a bottleneck in training and inference.

We benchmarked v4 vs v5 using:

- v4 branch: <https://github.com/ariG23498/transformers/tree/bench-v4>
- v5 branch: <https://github.com/ariG23498/transformers/tree/bench-v5>

Example:

```
from transformers import AutoModelForCausalLM

model_id = "Qwen/Qwen1.5-110B-Chat"
model = AutoModelForCausalLM.from_pretrained(model_id)
```

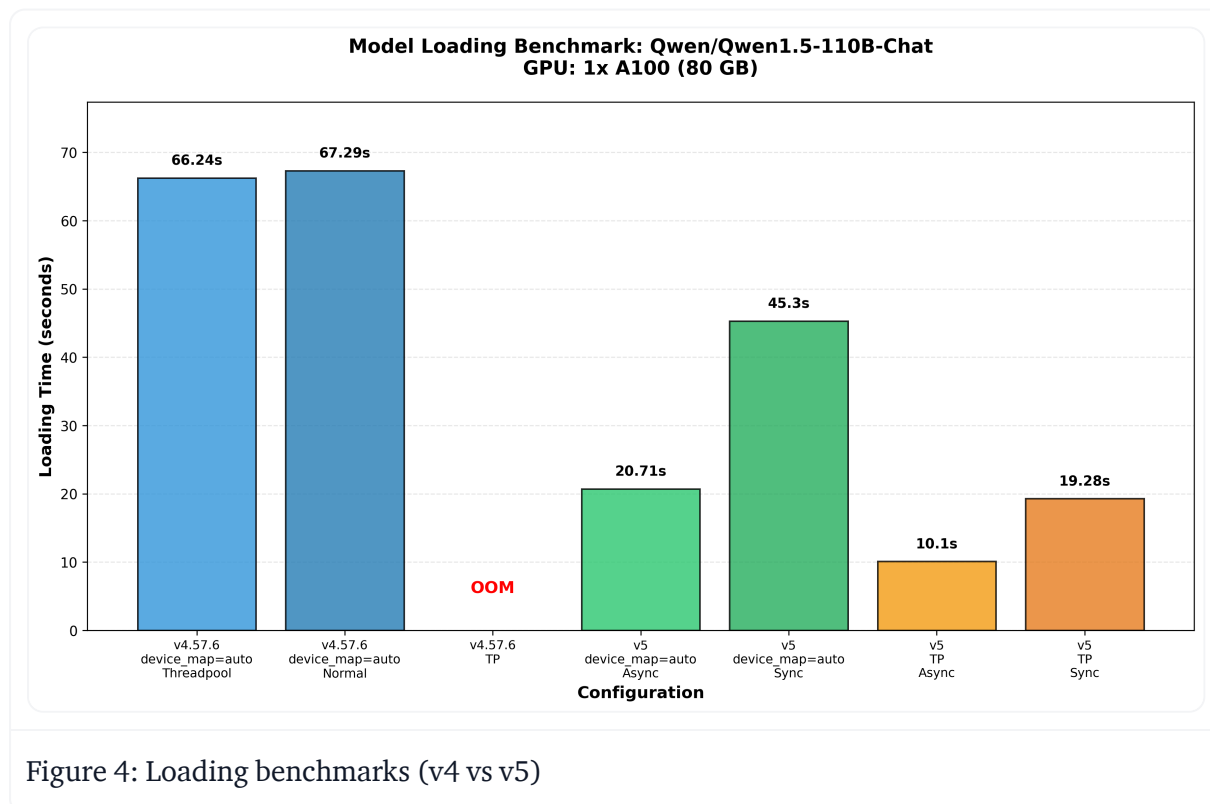
Two relevant environment variables:

- `HF_ENABLE_PARALLEL_LOADING`: Enables parallel shard loading via threads.
- `HF_DEACTIVATE_ASYNC_LOAD`: Disables the new async pipeline (v5 escape hatch).

[🔗](#) Results

Model: Qwen/Qwen1.5-110B-Chat **GPU:** 1× A100 (80GB)

Version	Strategy	Loading Mode	Time
v4.57.6	device_map="auto"	Threadpool	66.24s
v4.57.6	device_map="auto"	Sequential	67.29s
v4.57.6	TP	—	OOM
v5	device_map="auto"	Async (default)	20.71s
v5	device_map="auto"	Sync	45.3s
v5	TP	Async	10.1s
v5	TP	Sync	19.28s



The speedup is not just “more threads.”

It’s the combination of **Single-pass routing**, **Async materialization**, and **Conversion-aware scheduling** which together avoid unnecessary materialization and memory peaks while enabling expert packing and projection fusion at load time.

🔗 Where Quantization Fits In

With this refactor we can now create the runtime module structure first and then convert the weights into the structure. We can now optionally attach quantization within the conversion pipeline, making quantization part of the weight loading pipeline itself. This is crucial because quantizing “per expert” only makes sense once experts exist in a predictable packed layout.

This end to end pipeline was not possible earlier and now it comes to the users as an exposed API.

🔗 Expert Backend

Once experts are packed into a single runtime tensor, another question arises:

“How do you actually route through them efficiently?”

In a Mixture of Experts model, each token is routed to different experts. This means the runtime must dispatch tokens to their selected expert weights, execute the projections efficiently, apply the routing weights and then collect and reorder the results.

This is what the [Experts Backend system](#) (introduced in [PR #42697](#)) addresses. The Experts Backend introduces a **pluggable execution architecture** that decouples expert computation from the model implementation. Instead of hardcoding one dispatch strategy inside each MoE model, the system allows expert layers to dynamically select a backend at runtime.

This is implemented via a decorator pattern:

```
@use_experts_implementation
```

The decorator wraps expert classes and dispatches computation to the selected backend automatically.

Three backends are currently provided:

1. `eager` which loops over the selected experts and applies projections per expert. This is used for correctness reference and debugging.
2. `batched_mm` uses the `torch.bmm` API. This duplicate selected expert weights per token and performs a single batched GEMM. This backend is very well suited for small batch, GPU-heavy workloads where memory is available.
3. `grouped_mm` uses `torch._grouped_mm` API. Here we sort tokens by expert ID, group them, and then perform a single grouped GEMM. This backend shines with large batches or memory-constrained setups.

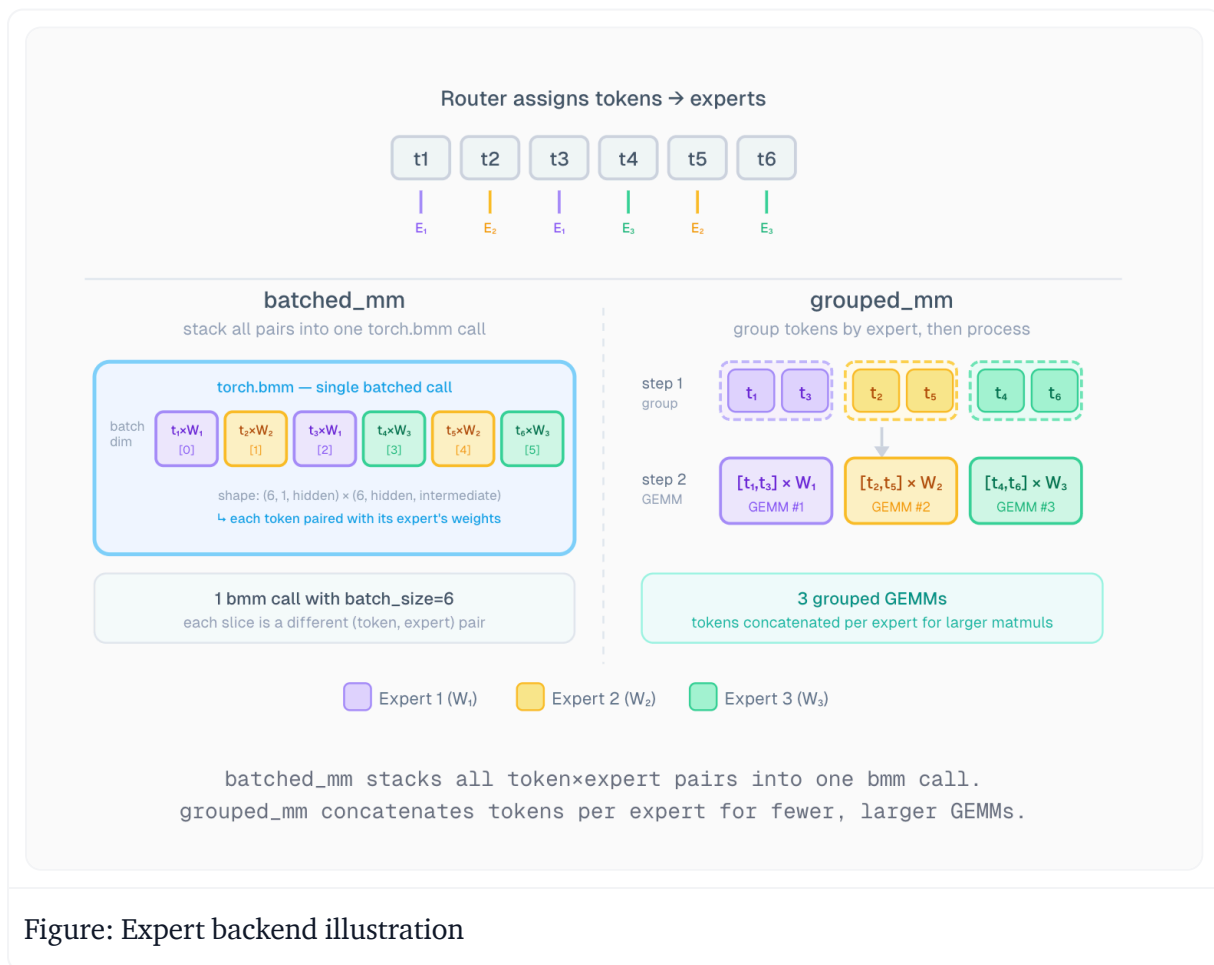


Figure: Expert backend illustration

🔗 Expert Parallelism

Mixture of Experts (MoE) models can have hundreds of billions of parameters (far more than what fits on a single GPU). Expert parallelism (EP) addresses this by distributing experts across multiple devices. Each device loads only its assigned subset of experts, computes for those experts and then participates in result aggregation. This approach scales models to far larger parameter counts without increasing computation cost because each token activates only a few experts.

Expert parallelism is enabled via `enable_expert_parallel`:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers.distributed.configuration_utils import DistributedConfig
```

```
distributed_config = DistributedConfig(enable_expert_parallel=True)

model = AutoModelForCausalLM.from_pretrained(
    "openai/gpt-oss-120b",
    dtype="auto",
    distributed_config=distributed_config,
)
```

Launch with:

```
torchrun --nproc-per-node N script.py
```

Where N evenly divides the total number of experts, and possibly matches the number of GPUs in your node.

When `enable_expert_parallel=True`, the model switches from the standard tensor-parallel (TP) plan to an expert-parallel (EP) plan with specialized sharding strategies.

Core components of EP lie in:

1. [GroupedGemmParallel](#): This splits the expert weights along the expert dimension (`dim=0`). Here each device loads only `num_experts / num_devices`.
2. [RouterParallel](#): This remaps global expert indices to local indices, masks out experts not assigned to the current rank, ensures each device computes only with its local experts and uses an all-reduce to combine partial outputs across devices.

🔗 Training MoEs with Transformers

MoEs are excellent for scaling inference, but training them is significantly more complex.

MoEs have a Massive parameter count, the distributed expert communication is complicated, there are routing in-stabilities that need to be handled. To address this, we collaborated with **Unsloth** to enable significantly faster Mixture-of-Experts training:

- $\sim 12\times$ faster MoE training
- $>35\%$ VRAM reduction
- $\sim 6\times$ longer context
- $12\text{--}30\times$ overall speedup compared to v4

We leverage the Expert Backend abstraction, standardize around PyTorch's `torch._grouped_mm` API and use custom Triton grouped-GEMM + LoRA kernels. Unsloth builds on top of the Transformers (and TRL) optimizations to push performance further.

For full details, we recommend reading: [Unsloth's official guide](#)

🔗 Conclusion

As sparse architectures continue to evolve, we want the transformers library to evolve with them. If you're building with MoEs or experimenting with new sparse ideas, we'd love to hear from you. Let us know what abstractions, kernels, or workflows you'd like to see next in `transformers`.

📦 Models mentioned in this article 6

🔗 [deepseek-ai/DeepSeek-R1](#)

📄 Text Generation • 📊 685B • Updated Mar 27, 2025 • ⬇️ 9.6M • ❤️ 13.5k

🔗 [deepseek-ai/DeepSeek-V2](#)

📄 Text Generation • 📊 236B • Updated Jun 8, 2024 • ⬇️ 10.4k • ❤️ 334

 **deepseek-ai/DeepSeek-V3**

 Text Generation •  685B • Updated Mar 27, 2025 •  1.18M •  4.11k

 **kernels-community/megablocks**

Updated Jun 4 •  2.56k •  5

 **mistralai/Mixtral-8x7B-v0.1**

 47B • Updated Jul 24, 2025 •  47.5k •  1.82k

 **openai/gpt-oss-20b**

 Text Generation •  22B • Updated Aug 26, 2025 •  8.5M •  4.87k

Papers mentioned in this article 2

Scaling Laws for Neural Language Models

 Paper • 2001.08361 • Published Jan 23, 2020 •  10

OLMoE: Open Mixture-of-Experts Language Models

 Paper • 2409.02060 • Published Sep 3, 2024 •  81

Collections mentioned in this article 5

MiniMax-M2  Collection

<https://arxiv.org/abs/2605.26494> • 4 items • Updated May 27 •  28

Qwen3.5  Collection

21 items • Updated Mar 9 •  1.74k

Kimi K2.5  Collection

Moonshot's large visual-language model • 4 items • Updated 7 days ago •  84

gpt-oss  Collection

Open-weight models designed for powerf... • 2 items • Updated Aug 7, 2025 •  464

GLM-5  Collection

2 items • Updated Feb 11 •  38



Vllm Transformers Inference

Native-speed vLLM transformers modeling backend



△ 65 July 8, 2026



Transformers Pytorch Optimization

Unlocking asynchronicity in continuous batching



△ 65 May 14, 2026

The `enable_expert_parallel` flag hiding the complexity of `GroupedGemmParallel` + `RouterParallel` behind a single config is a great DX win — distributing experts across devices used to require a lot of custom plumbing.



 Reply

Edit

Preview

Start discussing this article

 Tap or paste here to upload images

 Comment

[Sign up](#) or [log in](#) to comment

 System theme

Company

TOS

Privacy

About

Careers

Website

Models

Datasets

[Spaces](#)

[Pricing](#)

[Docs](#)

